

Servo Motor Implementation

[SG90]

➤ *Verilog HDL Design*

[Reference] ▪ <https://datasheetspdf.com/pdf/791970/TowerPro/SG90/1>

Micro Servo Module(SG90)

➤ *Basic and Concepts*

➤ Basic and Concepts

서보모터

- ✓ 서보모터(servomotor, servo motor)는 범용 기계와 비교해 보면 핸들을 돌리는 손에 해당하는 부분으로 머리에 해당되는 정보처리회로(CPU)의 명령에 따라 동작기계 테이블 등을 움직이게 하는 모터이다.
- ✓ 물체의 위치·방위·자세·회전 속도 등을 제어량으로 하고 목표치의 변화에 뒤따르도록 구성된 자동제어계를 서보기구(servo)라하며, 보통 피드백(feed back) 회로를 가지고 있어, 출력의 검출부·목표치와의 오차 증폭부·조작부·제어 대상 등으로 구성된다. 이러한 서보계의 조작부에 사용되는 것이 서보모터인데, 전기식(직류·교류)·유압식·전기유압식으로 된 것들이 있다.
- ✓ [Ref] https://ko.wikipedia.org/wiki/%EC%84%9C%EB%B3%B4_%EA%B8%B0%EA%B5%AC

Micro Servo Module(SG90)

➤ *Servo_Exam.xpr*

➤ Basic and Concepts ➔ HW Pin

➤ 핀 구성 : *Servo, Vcc, Gnd*

- *Servo* : 서보모터 제어 신호를 받는 핀(*PWM* 제어)
- *VDD(5V)* : 5V 전원과 연결
- *GND* : *BASYS3 PMOD*의 접지와 연결

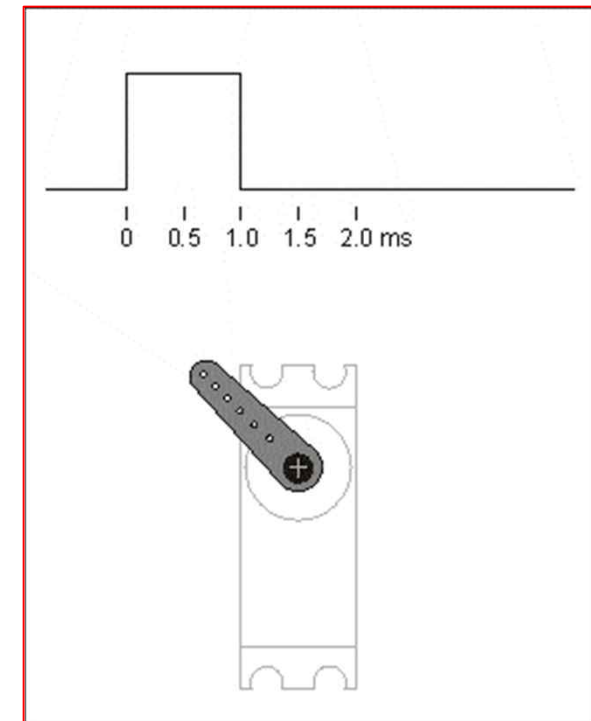
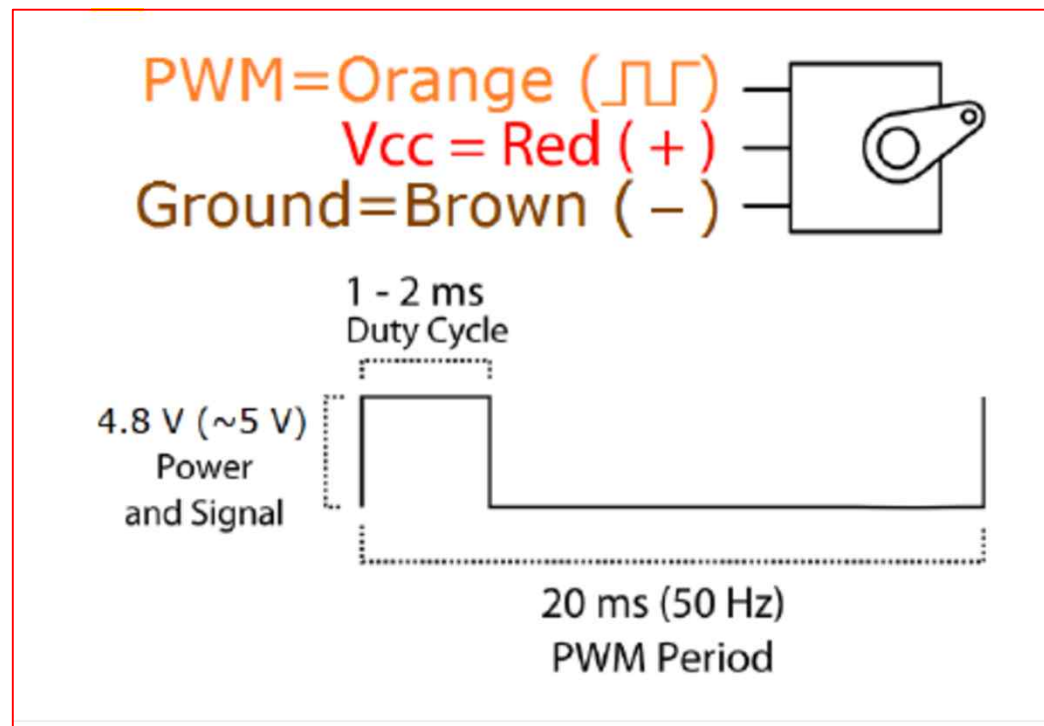


Micro Servo Module(SG90)

➤ Servo_Exam.xpr

➤ Basic and Concepts ➔ Micro Servo (SG90) 의 제어 원리

- PWM 파형의 *duty cycle*에 따라 각도가 변화.
- Data sheet 상으로 20ms 신호 중 1ms high에서 -90° , 2ms에서 $+90^\circ$
- 데이터 시트상의 각도와 실제 PWM 신호 인가시 각도가 다르므로, 직접 구동하면서 제어신호



Micro Servo Module(SG90)

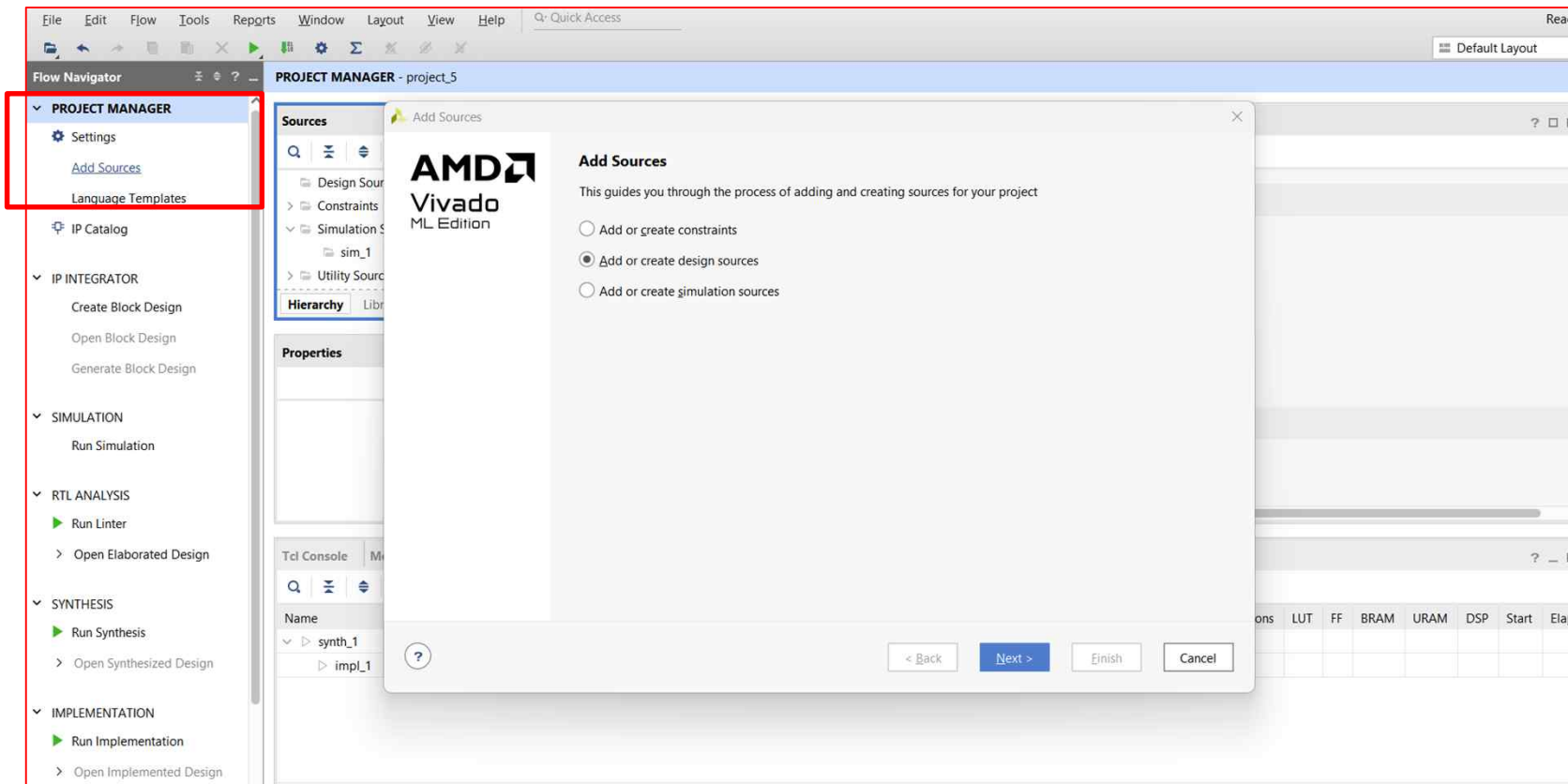
- *Basic and Concepts*
- *Module Verilog Code Design*

Micro Servo Module(SG90)

➤ *Servo_Exam.xpr*

➤ *Module Implementation*

- *PROJECT MANAGER → Add Sources*

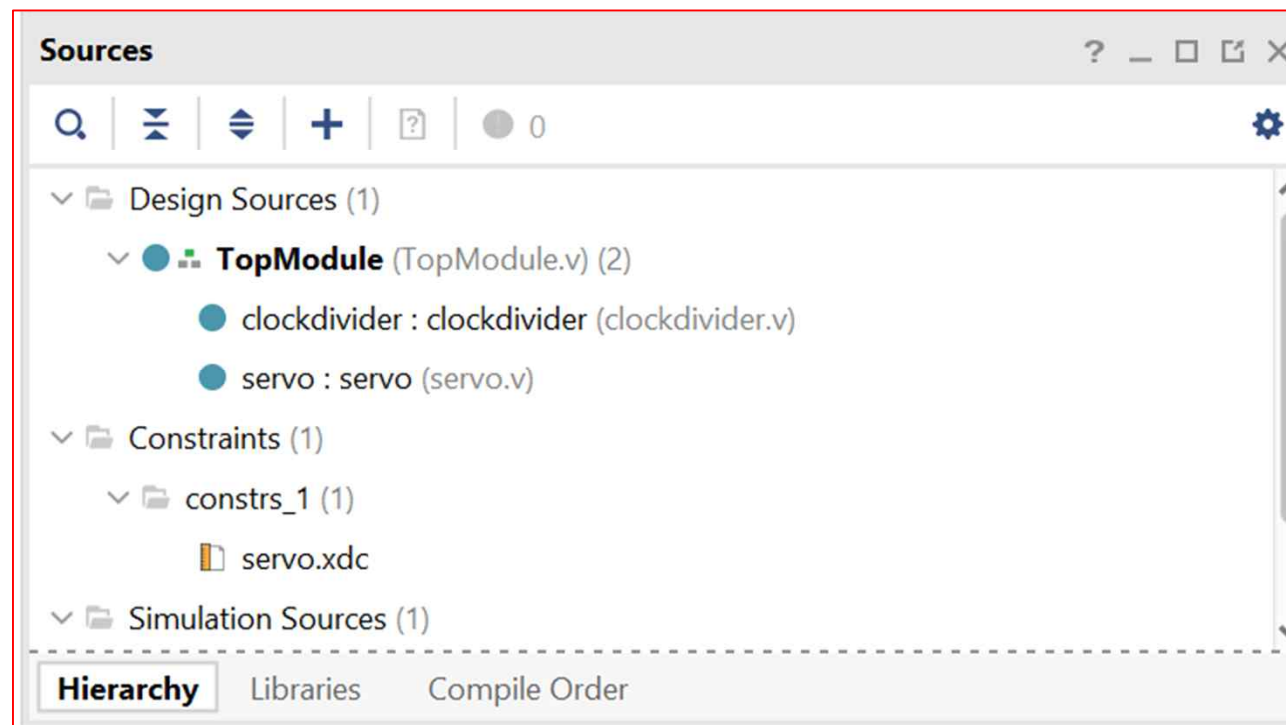


Micro Servo Module(SG90)

➤ *Servo_Exam.xpr*

➤ *Module Implementation*

- *PROJECT MANAGER* ➔ *Add Sources*
- *Create File* ➔ *servo.v, clockdivider.v, TopModule* 생성



Micro Servo Module(SG90)

➤ Servo_Exam.xpr

➤ **Module Implementation** ➔ Servo Motor Verilog HDL code Design

servo.v (1/2)

```
1. `timescale 1ns / 1ps
2. module servo(
3.     input clk, reset,
4.     input clk10000hz,
5.     input sw0,sw1,sw2,sw3,
6.     output reg Servo,
7.     output reg [1:0] mode
8. );
9.     reg [9:0] duty;
10.    reg [9:0] cnt_duty;
```

❖ 1-10: 입출력 port 및 레지스터 선언

- **clk**는 보드상에 장착된 **100Mhz**, **reset**은 **negedge reset**의 역할
- **clk10000hz** : clockdivider에서 **10khz clk(0.1msec)** 생성
- **mode** : 서보모터 제어 구분을 위해 선언함
- **Servo** : 서보모터 제어를 위한 **PWM** 신호

```
11.    always @(posedge clk10000hz or
12.        negedge reset) begin
13.        if(~reset) begin
14.            cnt_duty = 0;
15.            Servo = 0;
16.        end else begin
```

❖ 11-15: 리셋시 레지스터 초기화

Micro Servo Module(SG90)

➤ Servo_Exam.xpr

➤ **Module Implementation** ➔ Servo Motor Verilog HDL code Design

servo.v (2/2)

```
16. if(cnt_duty >= 200)
17.     cnt_duty = 0;
18. else
19.     cnt_duty = cnt_duty + 1;
20.
21. if(cnt_duty < duty)
22.     Servo = 1;
23. else Servo = 0;
24.
25. if(sw0 == 1'b1)
26.     mode = 2'b00;
27. else if(sw1 == 1'b1)
28.     mode = 2'b01;
29. else if(sw2 == 1'b1) mode = 2'b10;
30. else if(sw3 == 1'b1)
31.     mode = 2'b11;
32. else mode = 2'b00;
```

❖ 16 ~ 42: 서보모터 제어 신호 구현

- **cnt_duty : 200일 때 20ms**
- **Servo : 20ms 주기로 duty만큼 Servo 신호가 HIGH**

```
33. case(mode)
34. 2'b00 : duty <= 0;
35. 2'b01 : duty <= 7;
36. 2'b10 : duty <= 16;
37. 2'b11 : duty <= 26;
38. endcase
39.
40. end
41. end
42. endmodule
```

❖ 33~ 42 : duty 7 = 0.7ms (-90 °)
duty 16 = 1.6ms (0°)
Duty 26 = 2.6ms (+90°)

Micro Servo Module(SG90)

➤ Servo_Exam.xpr

➤ TopModule , clockdivider

TopModule.v

```
1. `timescale 1ns / 1ps
2. module TopModule(
3.   input clk,
4.   input reset,
5.   input sw0,sw1,sw2,sw3,
6.   output Servo,
7.   output [1:0] mode
8. );

9. clockdivider clockdivider(
10.   .clk(clk),
11.   .reset(reset),
12.   .clk10000hz(w_clkout)
13. );

14. servo servo(
15.   .clk(clk),
16.   .reset(reset),
17.   .clk10000hz(w_clkout),
18.   .sw0(sw0),
19.   .sw1(sw1),
20.   .sw2(sw2),
21.   .sw3(sw3),
22.   .Servo(Servo),
23.   .mode(mode)
24.
25. );
26. endmodule
```

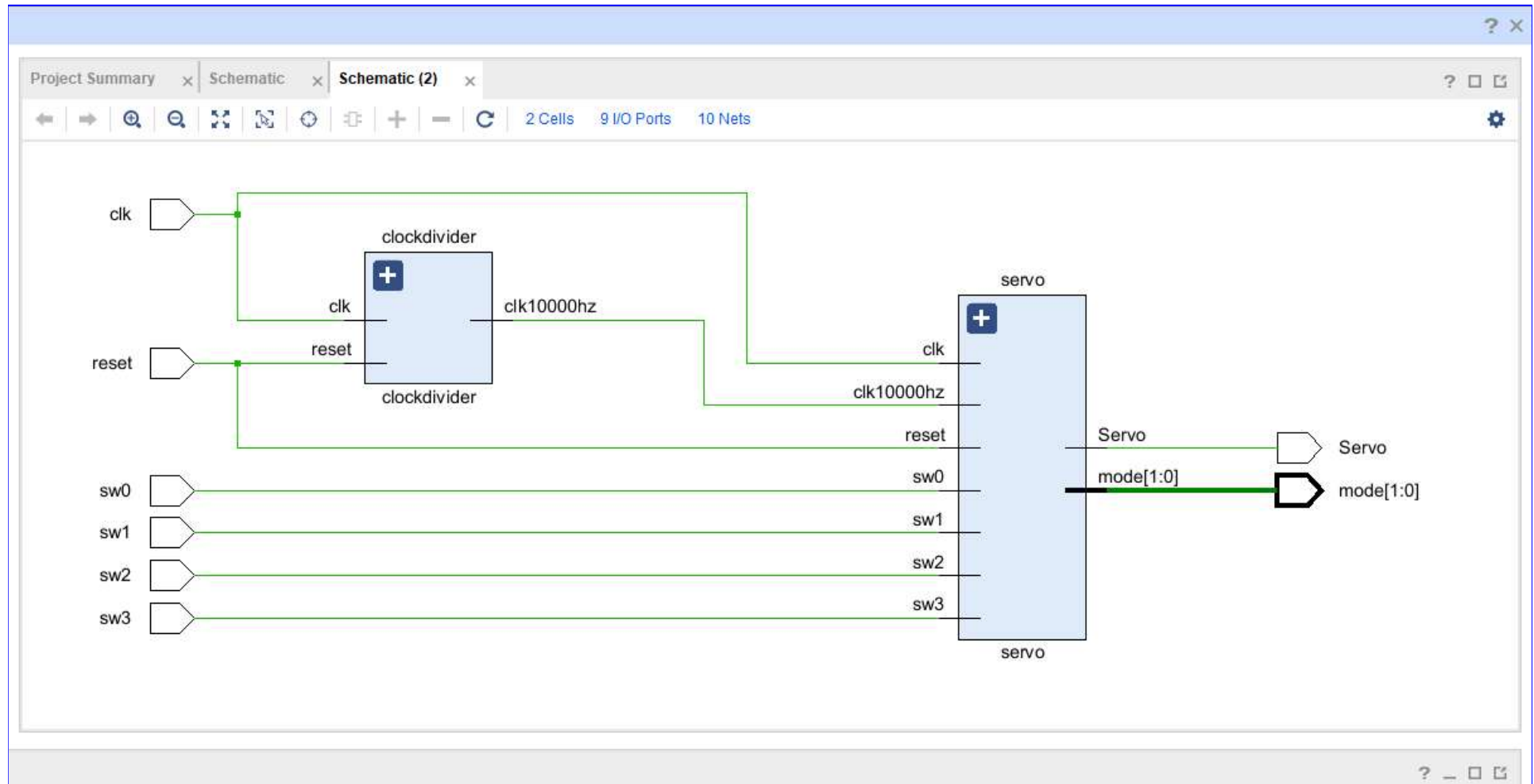
clockdivider.v

```
1. `timescale 1ns / 1ps
2. module clockdivider (
3.   input clk,
4.   input reset,
5.   output reg clk10000hz
6. );
7.   reg [25:0] cnt = 0;
8.   always @(posedge clk, negedge reset) begin
9.     if (~reset) begin
10.       cnt <= 0;
11.     end else begin
12.       if (cnt == (5_000 - 1)) begin
13.         cnt <= 0;
14.         clk10000hz <= ~clk10000hz;
15.       end else begin
16.         cnt <= cnt + 1;
17.       end
18.     end
19.   end
20. endmodule
```

Micro Servo Module(SG90)

➤ *Servo_Exam.xpr*

➤ *Micro Servo Modul* ➔ *RTL*



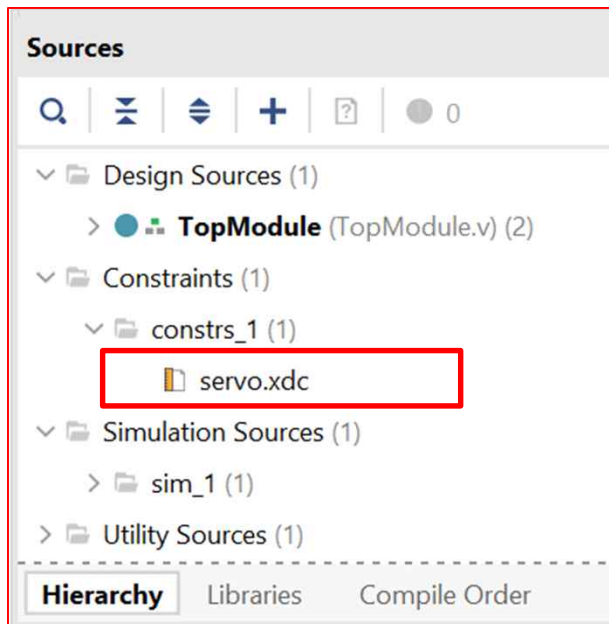
Micro Servo Module(SG90)

- *Basic and Concepts*
- *Module Verilog Code Design*
- *Constraints(XDC)*

Micro Servo Module(SG90)

➤ Servo_Exam.xpr

✓ xdc pin-mapping 완료 후 실습



```
set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
```

Switches

```
set_property -dict { PACKAGE_PIN R2 IOSTANDARD LVCMOS33 } [get_ports {reset}]
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports {sw0}]
set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports {sw1}]
set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports {sw2}]
set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports {sw3}]
```

LEDs

```
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports {mode[0]}]
set_property -dict { PACKAGE_PIN E19 IOSTANDARD LVCMOS33 } [get_ports {mode[1]}]
```

##Pmod Header JC

```
set_property -dict { PACKAGE_PIN K17 IOSTANDARD LVCMOS33 } [get_ports {Servo}];#Sch name = JC1
```

##USB-RS232 Interface

```
#set_property -dict { PACKAGE_PIN B18 IOSTANDARD LVCMOS33 } [get_ports usb_uart_rxd]
#set_property -dict { PACKAGE_PIN A18 IOSTANDARD LVCMOS33 } [get_ports usb_uart_txd]
```

Configuration options, can be used for all designs

```
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]
```

SPI configuration mode options for QSPI boot, can be used for all designs

```
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
```

Configuration options, can be used for all designs

```
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]
```

SPI configuration mode options for QSPI boot, can be used for all designs

```
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
```

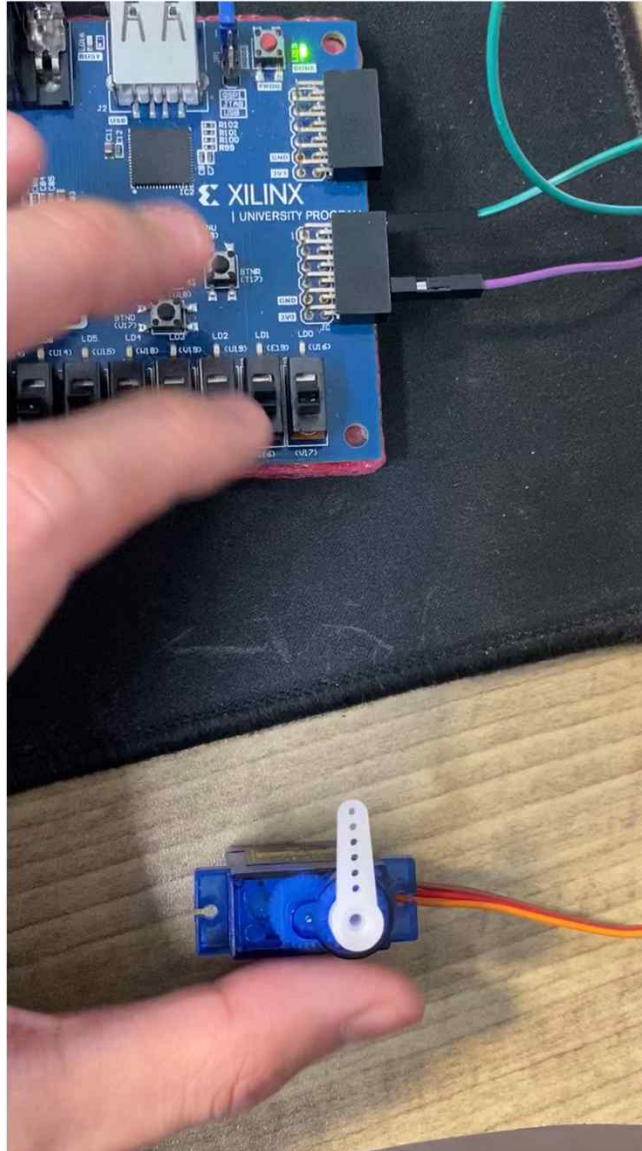
Micro Servo Module(SG90)

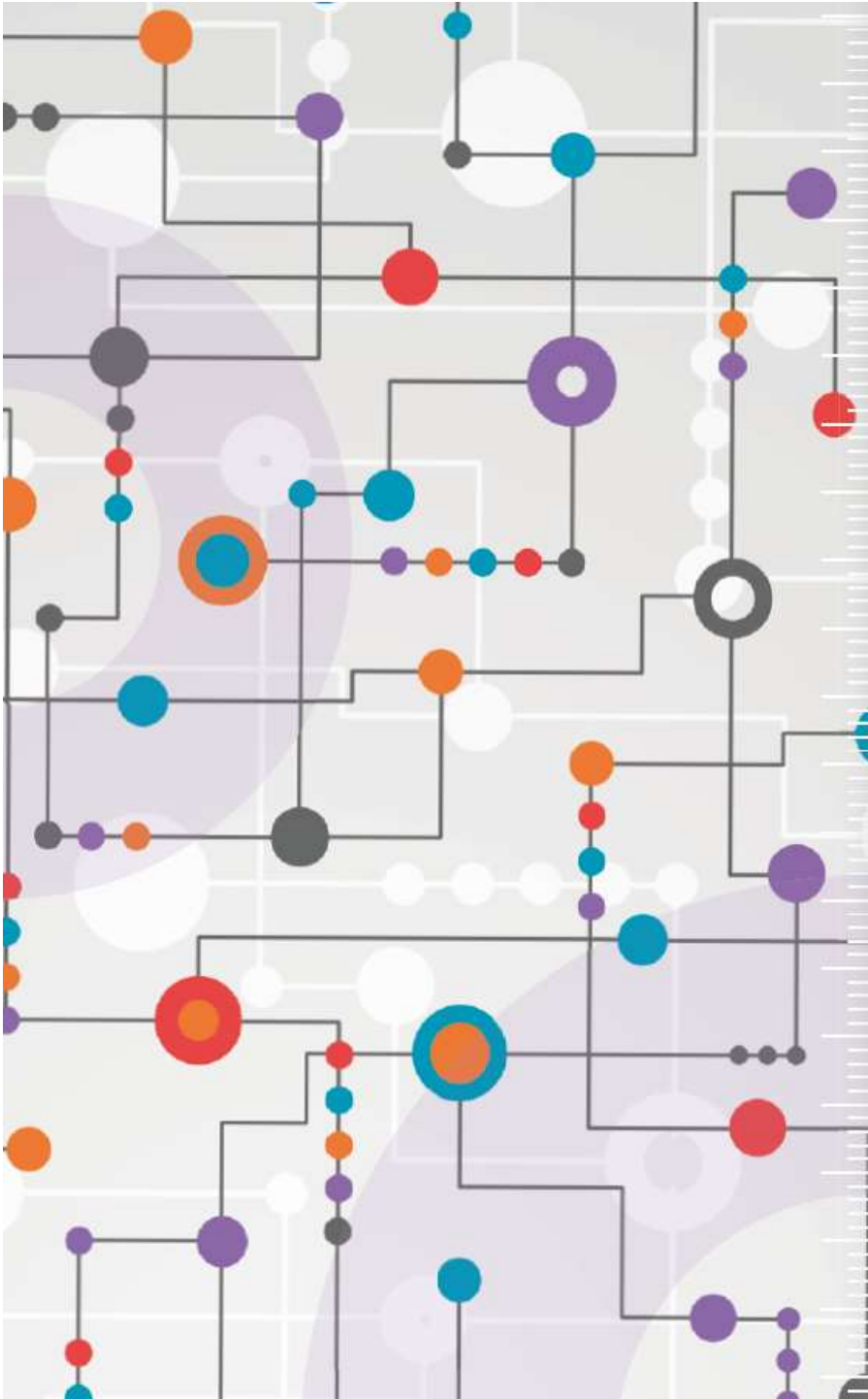
- *Basic and Concepts*
- *Module Verilog Code Design*
- *XDC*
- *Program Device and Check The Result*

Micro Servo Module(SG90)

➤ *Servo_Exam.xpr*

✓ *Program Device & Check The Result*





수고하셨습니다.